



Facultad de Estudios Superiores Acatlán

Diplomado en Ciencia de Datos

Proyecto final · Módulo V



Señal de arrepentimiento temprano
antes de comprar un videojuego

Fernando Axel Ramírez Gómez

30 de septiembre de 2026

Repositorio: <https://github.com/fernandoaxelramirezgomez-coder/nexplay>

App en vivo: <https://nexplay-six.vercel.app>



Resumen

Steam deja devolver un videojuego si se ha jugado menos de dos horas. NexPlay usa ese límite para ayudar antes de comprar: cuenta, en cada juego, las reseñas negativas escritas antes de las dos horas, una señal de que el juego decepcionó pronto. Con 123,972 reseñas de 83 juegos se entrenó un modelo sencillo que estima qué juegos dejan más de esa señal usando solo lo que cualquiera ve en la tienda antes de pagar: si el juego es gratis, su precio, su descuento y la nota de la crítica. El modelo distingue a esos juegos mejor que el azar, también en 40 juegos que no vio, aunque la ventaja es modesta. El sitio no da probabilidades: da una banda de riesgo, bajo, medio o alto; explica qué la mueve y de qué se quejan las reseñas; deja comparar juegos y preguntarle a un asistente, y recuerda la ventana de reembolso. Un análisis aparte, con modelos que leen las reseñas, mostró que esas quejas tempranas tienen un lenguaje propio: hablan de entrar al juego, del rendimiento y del reembolso. El riesgo, sin embargo, sale solo de datos del juego y es el mismo para cualquier persona.

Índice

Resumen	1
1. Introducción	2
2. Planteamiento del problema	2
3. Estrategia	3
4. Descripción de los datos	6
5. Análisis exploratorio	8
6. Procesamiento de datos	12
7. Modelación y evaluación	14
8. Resolución del problema	21
9. Conclusiones	23
Referencias	25
A. Reproducibilidad	25

1. Introducción

Comprar un videojuego en Steam tiene una salida: se puede pedir el reembolso dentro de las dos semanas posteriores a la compra, si se ha jugado menos de dos horas (Valve Corporation, 2024). Esa ventana es corta. Quien descubre que un juego no era lo que buscaba tiene un par de horas para darse cuenta y actuar, y si no lo hace, se queda con un gasto que no quería.

NexPlay estima, antes de pagar, qué tan seguido un juego deja a sus compradores arrepentidos en las primeras horas. No observa el arrepentimiento: usa una señal que sí está en los datos, las reseñas negativas escritas antes de las dos horas de juego, y la llama *señal de arrepentimiento temprano* (sección 5.1). El proyecto responde dos preguntas. La de datos: ¿qué tienen en común los juegos que dejan más señal, y se puede anticipar con lo que se sabe del juego antes de comprarlo? La de producto: ¿cómo se le muestra eso a quien duda entre dos juegos, sin prometerle una probabilidad que el modelo no da?

Entrega un modelo de riesgo por título, entrenado con 83 juegos de Steam y probado con 40 que no vio (sección 7); un sitio con el catálogo, la ficha de cada juego, una comparación y un asistente, Nia (sección 8); tres notebooks que corren en Colab (anexo A), y este documento. La tabla 1 lleva cada criterio de la rúbrica a la sección que lo cubre.

Criterio de la rúbrica	Dónde se cubre
Introducción	Resumen y sección 1
Planteamiento del problema	Sección 2
Estrategia: accionables, usuarios y usabilidad	Sección 3
Descripción de los datos	Sección 4
Análisis exploratorio de datos	Sección 5
Procesamiento de datos	Sección 6
Modelación supervisada	Sección 7
Resolución del problema	Sección 8
Interfaz de uso: paso a paso y demostración	Secciones 3.3 y 8.2
Conclusiones	Sección 9
Referencias en formato APA	Referencias
Reproducibilidad de los notebooks	Anexo A

Tabla 1: Cada criterio de la rúbrica y la sección que lo cubre.

2. Planteamiento del problema

La decisión antes de pagar. Quien compra un juego decide con lo que muestra la tienda: el precio, el descuento, las capturas, la nota de la crítica y el porcentaje de reseñas positivas. Lo que más le importa saber no está ahí: si el juego le va a servir en las primeras horas, que es cuando todavía lo puede devolver. Un juego que decepciona pronto cuesta

dinero si se deja pasar la ventana de reembolso, y tiempo y trámites si se aprovecha.

Por qué el promedio de reseñas no basta. El porcentaje de reseñas positivas junta a quien jugó diez minutos con quien jugó mil horas: una negativa escrita al poco rato y otra escrita tras cien horas cuentan igual. La pregunta de quien compra es más estrecha, y la señal de ese tramo temprano cambia mucho de un juego a otro (sección 5.3), así que vale la pena estimarla por título.

Qué se puede observar y qué no. Los datos dicen si una reseña fue negativa y cuánto había jugado su autor. No dicen si se arrepintió, si pidió el reembolso ni qué pasó con quien compró y no escribió (sección 5.1). Tampoco hay datos de PlayStation ni de Xbox: el lado del juego podría transferirse a otras tiendas, pero el riesgo que da el sitio se estimó solo con reseñas de Steam.

Nicho: una capa previa a la compra. NexPlay no vende juegos ni compete con la tienda. Es una consulta que se hace antes de pagar, con el enlace a Steam al lado, y su lugar está entre la página del juego y el botón de compra. Está pensado para quien compra desde México: los precios son los de la tienda de México.

El usuario, por comportamiento. El usuario no se define por edad ni por cuántos juegos tiene, sino por el momento: alguien a punto de comprar un juego para PC, que duda entre opciones o quiere saber a qué se arriesga. Los datos apoyan no definirlo por experiencia. Entre los perfiles públicos de data-v1, los veteranos, con 20 juegos o más, dejan la señal en el 2.89 % de sus reseñas, y los novatos, en el 0.67 % (como promedio por juego, 2.72 % contra 0.97 %). La diferencia se sostiene dentro de cada juego: la razón de Mantel-Haenszel¹ es 2.6, con un intervalo de 1.84 a 4.12. La señal no viene de la in-experiencia. Aun así, el riesgo que muestra el sitio es del título: la biblioteca que declara una persona no está validada, y su aporte al modelo cabe en el ruido (sección 7.2).

3. Estrategia

3.1. Usuarios y necesidades

Quien usa NexPlay está por comprar un juego en Steam desde México. Necesita tres cosas: saber antes de pagar si el juego tiende a decepcionar pronto, entender por qué, y decidir entre opciones con eso a la vista. El sitio le pide un perfil corto, que se declara en un formulario: compras al año, horas por semana, tolerancia a la fricción, las etiquetas de Steam que prefiere o rechaza y la plataforma. Ese perfil sirve para la afinidad, que dice qué tanto encaja un juego con quien lo declaró; no mueve el riesgo, que es del título (sección 7.2).

¹Compara a veteranos y novatos dentro de cada juego y combina las comparaciones; vale 1 si dentro de cada juego no hay diferencia.

3.2. Accionables

Cada resultado del proyecto lleva a una acción concreta en el sitio:

- **Comparar antes de decidir.** Comparar pone lado a lado la banda de riesgo, el motivo principal, el precio y la crítica de los juegos elegidos (figura 2a).
- **Revisar si las quejas son de las que le afectan.** Los motivos dicen de qué se quejan las reseñas con la señal; un juego cuyas quejas son de rendimiento pesa distinto para quien tiene un equipo de sobra que para quien no (figura 1e y f).
- **Preguntarle a Nia,** que contesta con los datos del catálogo y explica la banda de un juego (sección 8.2).
- **Planear las dos primeras horas.** El Inicio recuerda la ventana de reembolso y lo que más mencionan las negativas tempranas, para probar eso primero mientras el juego todavía se puede devolver; Steam decide cada solicitud, y el sitio lo dice (figura 2c).
- **Buscar alternativas del mismo género con riesgo bajo.** Explorar agrupa el catálogo en estantes por banda y se filtra por género (figura 2b).

3.3. Recorrido: PAYDAY 3 contra Dead Space

Una persona duda entre PAYDAY 3 y Dead Space, dos juegos de acción con precios casi iguales al corte de datos: 344.00 pesos PAYDAY 3, al 14 de septiembre de 2026, y 349.75 Dead Space, al 21 de septiembre de 2026, con 75 % de descuento sobre 1,399.00. Son los precios que habría visto quien comprara esos días. El recorrido tiene cinco pasos:

1. En el Inicio busca los dos. La ficha de PAYDAY 3 dice riesgo alto y la de Dead Space, riesgo bajo, cada una con su frase (figura 1a y b). Dead Space es uno de los 40 juegos que el modelo no vio al entrenar.
2. «Qué mueve esta estimación» da los factores (figura 1c y d). PAYDAY 3 tiene una nota de Metacritic de 66, por debajo del promedio del catálogo, y eso sube su riesgo estimado; Dead Space tiene 87. El descuento de Dead Space aparece como un factor que lo baja, pero con evidencia débil, así que no es parte del argumento.
3. Los motivos marcan la diferencia (figura 1e y f). En PAYDAY 3, la queja principal es el contenido, en el 48 % de las 33 reseñas clasificadas de sus 113 con la señal. En Dead Space es el rendimiento, en el 71 % de 45, de 80. Las quejas de rendimiento pesan según el equipo de quien juega; las de contenido, no.
4. Comparar los pone lado a lado (figura 2a), y Nia puede explicar la banda de cualquiera de los dos.
5. Si quiere otro juego de acción con riesgo bajo, Explorar muestra un estante de 25 (figura 2b). Va del score más bajo al más alto, y el precio 0 empuja al frente a los gratis y a los que no tienen precio: Grand Theft Auto V Legacy lleva el aviso de precio imputado, y Team Fortress 2 y Apex Legends, el de juego gratis. La alternativa sin aviso es Resident Evil 4.

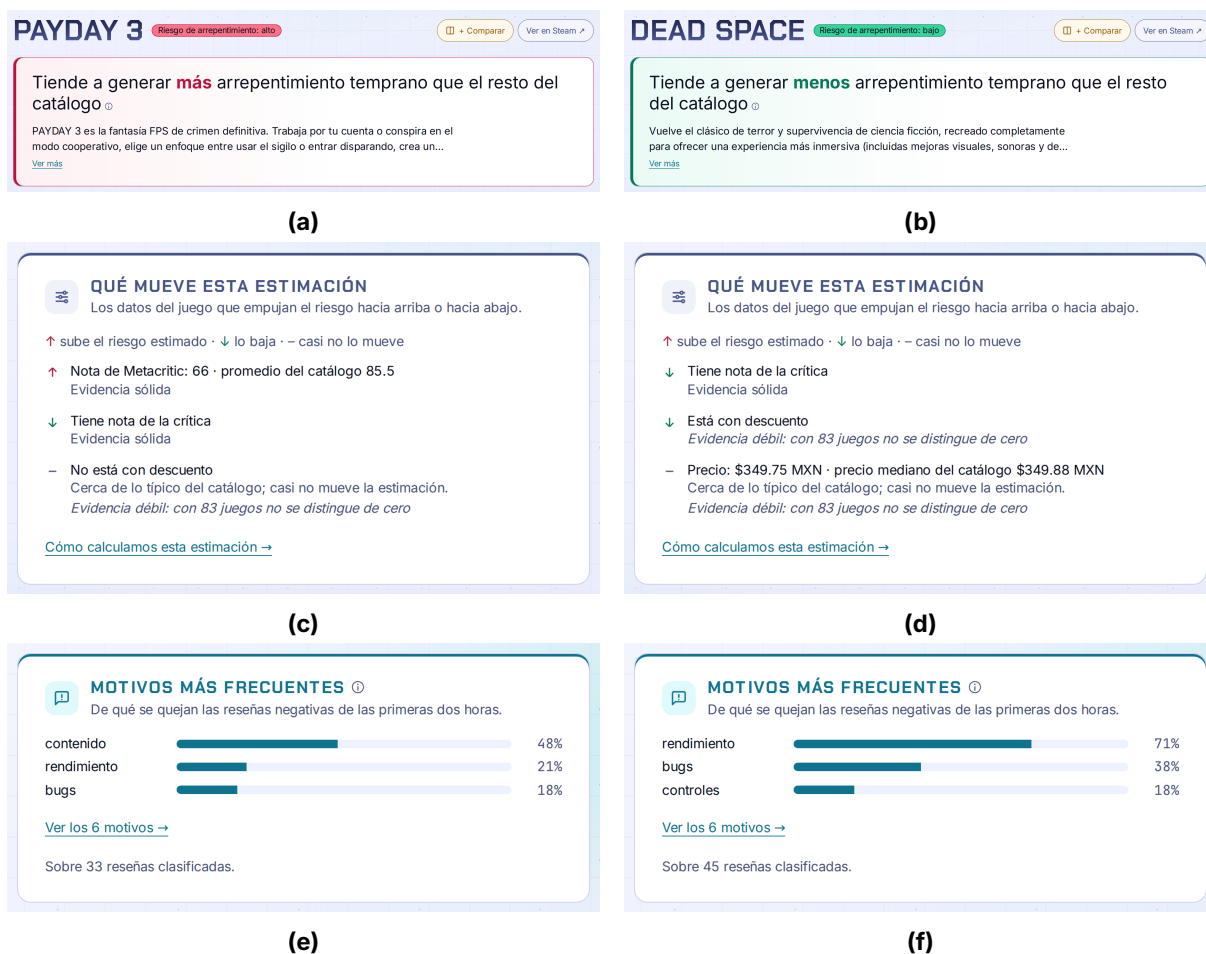


Figura 1: El recorrido en la ficha: PAYDAY 3 a la izquierda y Dead Space a la derecha. La banda (a, b), lo que mueve la estimación (c, d) y los motivos más frecuentes (e, f). Capturas provisionales del sitio: se regeneran desde el tag de entrega.

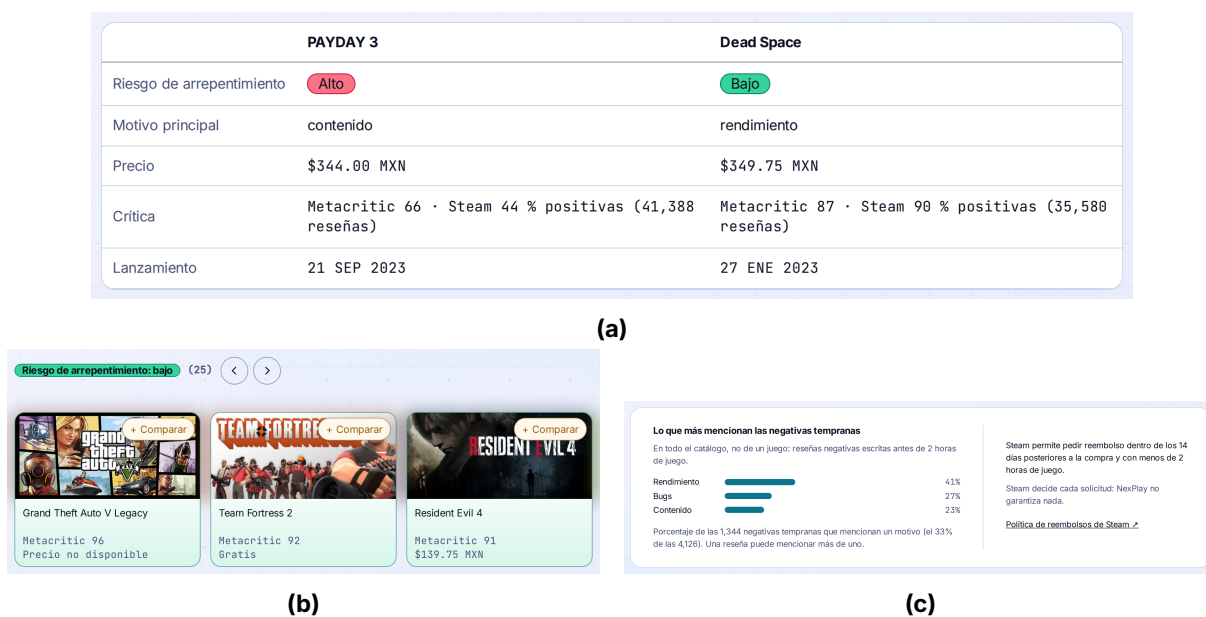


Figura 2: Tres accionables: Comparar (a), el estante de riesgo bajo de Acción en Explorar (b) y la ventana de reembolso en el Inicio (c), cuyos motivos se cuentan sobre todo el catálogo servido. Capturas provisionales del sitio: se regeneran desde el tag de entrega.

3.4. Usabilidad

La banda se dice con palabras y color, no con un porcentaje: el score no es una probabilidad calibrada (sección 7.4), y un porcentaje se leería como una. La ficha explica la banda con sus tres factores principales, cada uno con su nivel de evidencia, y los motivos con el número de reseñas sobre el que se calcularon, para que una cifra sobre pocas reseñas no parezca más firme de lo que es. Todo lo que muestra el riesgo es igual para cualquier persona, así que dos personas que consultan el mismo juego ven lo mismo. Los controles pequeños miden al menos 44 píxeles de alto, para tocarlos con el dedo en un teléfono, y el sitio tiene tema claro y oscuro.

4. Descripción de los datos

Los datos son reseñas públicas de Steam y la ficha de cada juego en la misma tienda. En cada reseña, Steam guarda si el voto fue negativo y cuántos minutos llevaba jugados quien la escribió; con esas dos cosas se define la señal de arrepentimiento temprano (sección 5.1). El modelo se entrena con 83 juegos y 123,972 reseñas (data-v1), y el sitio sirve un catálogo de 123 juegos y 184,367 reseñas (data-v3). No se usó Kaggle ni ningún conjunto de terceros.

4.1. La API de Steam

La ingesta, el proceso que bajó los datos de Steam, usa dos servicios públicos de la tienda². `appreviews` da, por reseña, el voto, los minutos jugados al escribir y en total, el texto, las fechas y dos datos del autor: cuántos juegos tiene y cuántas reseñas ha escrito. Se pidieron las más recientes, en inglés y de cualquier forma de compra, hasta 1,500 por juego (Valve Corporation, s.f.-b). Hoy Valve marca ese servicio como obsoleto y remite a `GetAppReviews` (Valve Corporation, s.f.-a), pero el proyecto no depende de volver a bajarlo. `appdetails`, que Valve no documenta de forma oficial, da por juego el precio y el descuento en pesos mexicanos, los géneros, la fecha de lanzamiento y la nota de Metacritic³ tal como la muestra la tienda: el proyecto nunca consulta Metacritic. En los 40 títulos externos, la nota guardada coincide con la de la tienda en 40 de 40⁴.

4.2. Releases y periodo

Cada corte de datos se publica como un release de GitHub con un tag fijo⁵, y antes de usar un archivo se compara su sha256 con el publicado⁶. Con data-v1 se toman todas las

²Una API (interfaz de programación) es una dirección web que devuelve datos en un formato que un programa puede leer, en lugar de una página pensada para personas. Estos dos servicios no piden clave.

³Metacritic promedia las calificaciones de la crítica especializada en una nota de 0 a 100. Steam muestra esa nota en la página de algunos juegos.

⁴La comparación está en `docs/evidencia/verificacion-40-steam.csv`.

⁵Un release es una versión publicada del repositorio, con archivos adjuntos. Su tag es el nombre que la identifica, como `data-v1`, y no cambia: quien lo baje dentro de un año recibe los mismos archivos.

⁶El sha256 es una huella de 64 caracteres que se calcula sobre el contenido de un archivo; si cambia un solo byte, cambia la huella. La comparación la hace `descargar_verificado`, en

decisiones del modelo: las variables, el entrenamiento y los umbrales. El release data-v2 suma 40 juegos que el modelo nunca ve, y data-v3 es lo que sirve el sitio. Ningún release guarda el identificador de Steam de quien escribió la reseña; el anexo A da cada archivo con su sha256.

Las reseñas de data-v1 se bajaron el 14 de septiembre de 2026, y las de los juegos que agregó data-v2, hasta el 21 de septiembre de 2026, en UTC⁷. La ingesta pide las más recientes hasta el tope, y 81 de los 83 juegos lo alcanzan: un juego popular lo llena en pocos días y uno de nicho necesita años, así que la muestra de cada juego cubre de 4 a 1,252 días, con una mediana de 64. Por eso el 96.1 % de las reseñas de entrenamiento es de julio de 2025 en adelante, con una cola que llega hasta el 8 de abril de 2023 (detalle en el notebook 00, §3.1). La muestra retrata el presente de cada juego: en uno viejo, la señal temprana viene de quien lo compra tarde. Es una limitación de la fuente (sección 9.1) y no se corrige con pesos, porque no hay forma de saber cómo era la mezcla de compradores en otra época.

4.3. Calidad

Antes de explorar, el notebook 00 valida data-v1: tipos y nulos, llaves, rangos, textos repetidos y la partición de validación, y dice qué es cada una de las 41 columnas de las dos tablas (detalle en el notebook 00, §1). Las llaves y los rangos imposibles no tienen un solo caso. Las dos columnas que definen la señal, el voto y los minutos al reseñar, no tienen nulos, y todas las variables del juego que usa el modelo se conocen antes de comprar. Tres hallazgos tocan al modelo, y ninguno se rellena en silencio. El precio de los 2 juegos de pago que llegaron sin precio se toma como 0, y la ficha lo avisa. La falta de nota de los 23 juegos sin Metacritic entra al modelo como una variable propia. Y el 0 de los perfiles privados⁸, que son el 59.9 % de las reseñas, se trata como una bandera de privacidad y no como una biblioteca vacía. Los textos repetidos se marcan y se conservan (sección 6).

El tope de 1,500 reseñas por juego es una decisión de diseño: así cada título pesa casi lo mismo en el entrenamiento, y unos cuantos juegos muy populares no dominan la muestra. Tiene otra consecuencia, los empates de tamaño entre juegos, que se trata con la validación (sección 7.3).

4.4. Por qué estos datos sirven para la pregunta

NexPlay quiere saber, antes de la compra, qué tan seguido un juego deja a sus compradores arrepentidos en las primeras horas. Las reseñas las escriben personas que tienen el juego, y cada una dice cuántos minutos llevaba jugados su autor: con eso se sabe si cae dentro o fuera de la ventana de reembolso de dos horas. `appdetails` da lo que cualquiera ve en la tienda antes de pagar, como el precio, el descuento y la nota de la crítica, y de ahí salen las variables del modelo. También tienen límites (sección 9.1): las reseñas

`backend/despliegue/utilidades.py`.

⁷UTC es el tiempo universal coordinado. La Ciudad de México va seis horas atrás.

⁸En Steam, cada persona decide si su perfil es público. Un perfil privado oculta la biblioteca, y la API lo reporta con 0 juegos.

son en inglés y las escriben compradores de todo el mundo, mientras que los precios son los de la tienda de México.

5. Análisis exploratorio

La sección define primero qué se predice y con qué métrica se mide. Después se queda con los hallazgos del notebook 00 que cambiaron una decisión; el resto del análisis está en el notebook.

5.1. La variable objetivo

Para cada reseña i ,

$$Y_i = \begin{cases} 1 & \text{si } \text{playtime_at_review}_i < 120 \text{ y } \text{voted_up}_i = 0, \\ 0 & \text{en otro caso,} \end{cases}$$

donde `playtime_at_review` son los minutos que llevaba jugados quien escribió la reseña cuando la publicó y `voted_up` es el pulgar: 1 arriba, 0 abajo⁹. En `data-v1` hay 2,714 reseñas con $Y = 1$ de 123,972, una prevalencia¹⁰ de 2.19 % por reseña. A esa etiqueta se le llama *señal de arrepentimiento temprano*. La tabla 2 resume los dos cortes de datos.

	Entrenamiento (data-v1)	Catálogo (data-v3)
Juegos	83	123
Reseñas	123,972	184,367
Reseñas con la señal ($Y = 1$)	2,714	4,126
Prevalencia por reseña	2.19 %	2.24 %
Reseñas publicadas	8 abr 2023 a 14 sep 2026	8 abr 2023 a 21 sep 2026
Publicadas desde julio de 2025	96.1 %	97.1 %
Reseñas con perfil público	40.1 %	40.2 %
Juegos gratis	2	7
Precio de los de pago en pesos: mediana (mín. a máx.)	400.00 (59.00 a 1,599.00)	349.88 (56.99 a 1,599.00)
Nota de Metacritic: mediana (mín. a máx.)	88 (61 a 97)	86 (61 a 97)
Juegos sin nota de Metacritic	23	33

Tabla 2: Estadísticas descriptivas del corte de entrenamiento y del catálogo que sirve el sitio. El precio y la nota son de los juegos, no de las reseñas. Fuente: releases `data-v1` y `data-v3`.

Por qué es una señal proxy. Steam no registra arrepentimiento: registra que alguien votó en contra y cuánto había jugado al escribir, así que la señal es una proxy¹¹. Los 120

⁹ Al reseñar, Steam pide elegir entre «Recomendado» y «No recomendado».

¹⁰ La proporción de casos positivos; aquí, de reseñas con $Y = 1$.

¹¹ Una medida que sí se puede observar y que se usa en lugar de otra que no se puede observar directamente.

minutos vienen de la política de reembolsos de la tienda: se puede pedir el reembolso dentro de las dos semanas posteriores a la compra y con menos de dos horas de juego (Valve Corporation, 2024). Una reseña negativa escrita dentro de esa ventana es lo más cercano que dejan los datos a «esto no era para mí, y me di cuenta pronto». Por eso el sitio y este documento la llaman señal y nunca dicen que alguien se arrepintió. La señal deja fuera a quien se arrepiente y no escribe, a quien pide el reembolso sin reseñar y a quien reseña después de las dos horas.

El umbral de 120 minutos. El umbral no se eligió con los datos, y moverlo casi no cambia qué juegos tienen más señal: con 60, 90 o 180 minutos, la correlación de Spearman¹² del orden de los juegos con el de 120 va de 0.96 a 0.99 (detalle en el notebook 00, §3.3). Lo que sí muestran los datos es que las negativas llegan antes: el 15.6 % de las reseñas negativas se escribe antes de los 120 minutos, contra el 2.0 % de las positivas (figura 3, en escala logarítmica¹³). En los 120 minutos no hay escalón: por minuto, las negativas pasan de 11.9 a 12.2 al cruzarlos, y las positivas, de 17.4 a 18.6. El único salto, a los 180 minutos y solo en las positivas, no toca la señal.

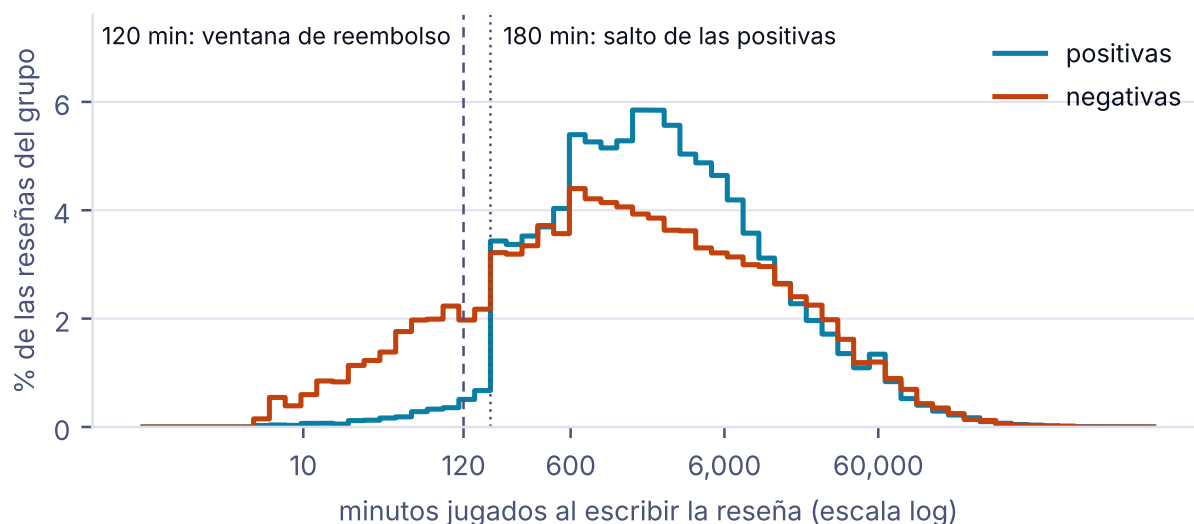


Figura 3: Minutos jugados al escribir la reseña, por pulgar, en escala logarítmica. Cada curva se normaliza sobre su propio grupo; los intervalos tienen un borde en 180 y ninguno en 120. Fuente: data-v1.

5.2. Por qué PR-AUC

Con una prevalencia de 2.19 %, un modelo que siempre dice «no» acierta el 97.81 % de las veces y no encuentra un solo caso, así que la exactitud no sirve. El proyecto usa el área bajo la curva de precisión y exhaustividad (PR-AUC)¹⁴, que con clases desbalan-

¹²Compara dos órdenes, no dos valores: vale 1 cuando los juegos quedan en el mismo orden, aunque sus tasas cambien.

¹³Cada marca del eje multiplica a la anterior por la misma cantidad.

¹⁴La precisión es la parte de lo que el modelo pone arriba que sí tiene la señal; la exhaustividad, la parte de los casos con señal que encuentra. El PR-AUC es el área bajo la curva que traza una contra la otra.

ceadas informa más que la curva ROC¹⁵ (Davis & Goadrich, 2006; Saito & Rehmsmeier, 2015). El PR-AUC de un clasificador que no sabe nada, que da a todos los casos la misma calificación, es la prevalencia: 0.0219. Ese es el piso, y por eso el PR-AUC del modelo siempre va junto al del trivial. En la validación agrupada por juego¹⁶ el modelo llega a 0.0694, 3.2 veces ese piso (sección 7.5). Esos cocientes contra el trivial llevan un decimal, porque el PR-AUC cambia bastante de un fold a otro y una centésima más no diría nada; los extremos de un intervalo llevan dos, porque ahí importa qué tan cerca quedan del 1, el valor en el que el modelo no ordena mejor que el trivial. Todo PR-AUC lleva tres cifras significativas, y su desviación, su intervalo o una diferencia, los mismos decimales que él. Los scores y los umbrales de las bandas llevan cuatro decimales, porque los juegos al filo de un corte se distinguen justo en el cuarto (sección 7.6).

5.3. La tasa varía por juego

¿La señal es parecida en todos los juegos, o depende del juego? La figura 4 ordena los 83 juegos de data-v1 según su tasa de señal, la proporción de sus reseñas con $Y = 1$, con su intervalo de Wilson¹⁷. La tasa va de 0.13 % a 27.13 %, y como hasta el juego con menos reseñas tiene 682, los intervalos son cortos al lado de esa distancia. No es ruido: la varianza de la tasa entre juegos es unas 107 veces la esperada si todos tuvieran la misma tasa¹⁸.

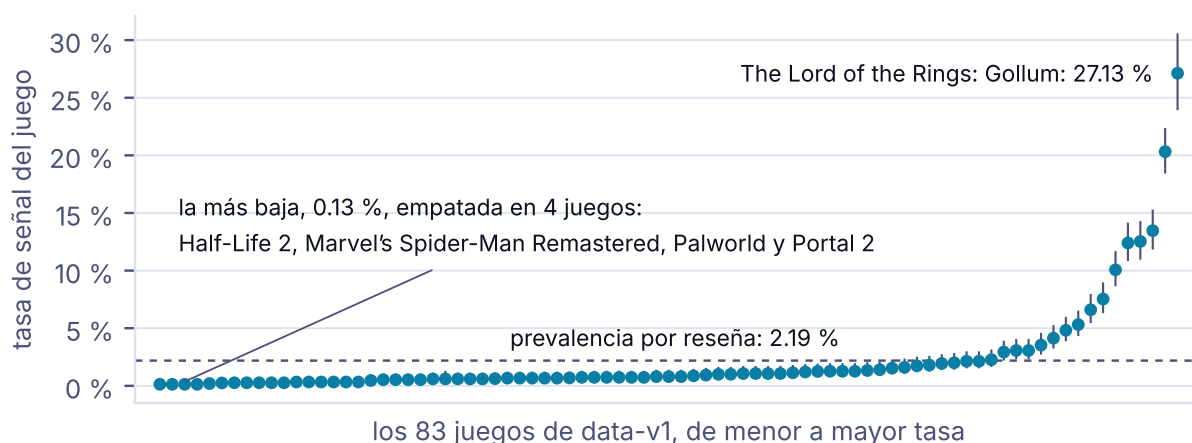


Figura 4: Tasa de señal de cada juego de data-v1, de menor a mayor, con su intervalo de Wilson al 95 %. La línea punteada es la prevalencia por reseña. Fuente: data-v1.

Además, las cinco variables del modelo valen lo mismo en todas las reseñas de un juego: el modelo solo puede aprender qué juegos tienen más señal. De ahí salen dos decisiones.

¹⁵Compara aciertos contra falsas alarmas. Con muchísimos negativos, las falsas alarmas pesan poco y la curva puede verse bien aunque el modelo señale muchos casos de más.

¹⁶GroupKFold reparte juegos completos entre entrenamiento y prueba, así que mide al modelo con juegos que no vio.

¹⁷Un intervalo de confianza marca entre qué valores es razonable que esté la tasa real. El de Wilson (1927) nunca baja de cero, y por eso sirve con tasas bajas.

¹⁸Con una sola tasa para todos, la de cada juego variaría solo por las reseñas que le tocaron, como en una binomial. El cociente divide la desviación al cuadrado de cada juego entre esa varianza esperada y promedia: es la ji cuadrada de Pearson entre sus grados de libertad, y vale cerca de 1 si solo el muestreo separa a los juegos.

La validación agrupa por juego, porque con las reseñas de un juego a la vez en entrenamiento y en prueba el modelo se mediría reconociendo juegos que ya vio (sección 7.3). Y las variables del juego se analizan con un punto por juego, porque 123,972 filas que repiten los mismos valores darían una precisión que no existe.

5.4. La crítica acompaña a la señal

¿Qué tienen los juegos con más señal? Juego por juego, solo la crítica tiene una correlación de Spearman con la tasa que difícilmente sale del azar¹⁹: -0.40 con tener nota y -0.42 con la nota, entre los juegos que la tienen, las dos con valores p por debajo de una milésima. Con el precio es 0.18 , con un valor p de 0.12 , y con el descuento tampoco se distingue del azar; la gratuidad casi no se puede medir, con 2 juegos gratis (detalle en el notebook 00, §3.4). En la figura 5, los 23 juegos sin nota tienen una tasa mediana de 2.13% , contra 0.73% de los que sí la tienen, y entre estos la nube baja conforme sube la nota, con mucha dispersión.

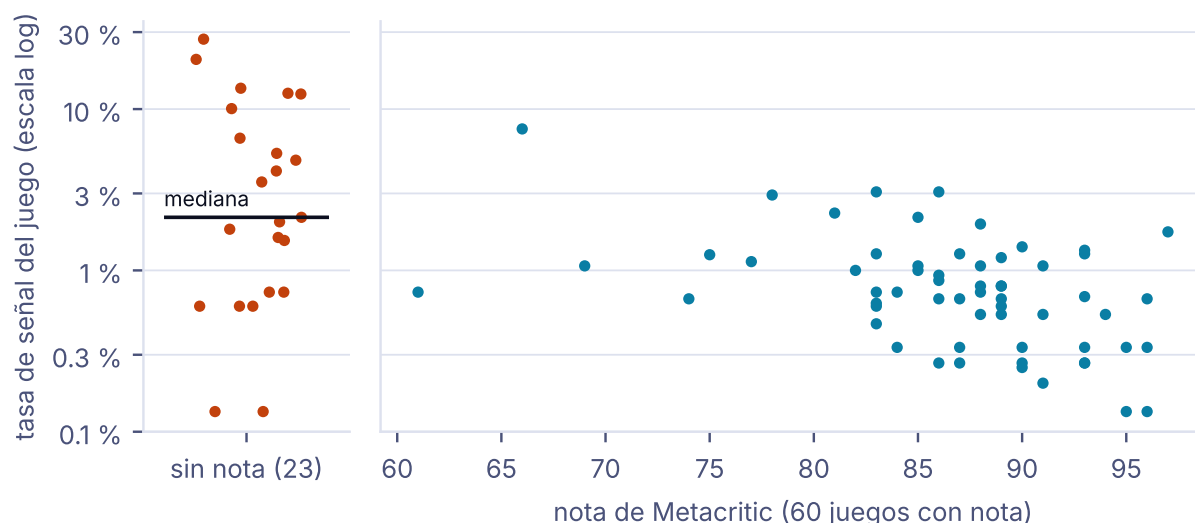


Figura 5: Tasa de señal de cada juego de data-v1 contra su nota de Metacritic, en escala logarítmica. Los juegos sin nota van en un panel aparte, desplazados al azar a lo ancho (con una semilla fija); la línea negra es su mediana. Fuente: data-v1.

Una variable a la vez puede engañar: si los juegos caros fueran también los sin nota, el precio parecería importar por culpa de la crítica. Con las cinco juntas, los coeficientes del modelo y un intervalo por bootstrap sobre juegos²⁰, solo los intervalos de la crítica no cruzan el cero, y los dos coeficientes son negativos: tener nota y tener una nota más alta bajan el riesgo estimado del título. El de tener nota va de -1.04 a -0.55 , y el de la nota, de -0.56 a -0.07 . El modelo se queda con las cinco variables, que se conocen antes de comprar y valen igual para cualquier persona; lo que cambió fue la explicación, porque cada factor del sitio lleva su nivel de evidencia, sólida para la crítica y débil para el resto (sección 7.7).

¹⁹El valor p es la probabilidad de ver una correlación así de fuerte si en realidad no hubiera ninguna.

²⁰El bootstrap repite el ajuste con muestras de juegos sacadas al azar y con reemplazo; el intervalo reúne el 95% central de los coeficientes. Las variables están estandarizadas, así que los coeficientes se comparan entre sí.

5.5. El texto de las negativas tempranas

En el conjunto limpio (sección 6.1) hay 105,885 positivas, 14,353 negativas tardías y 2,706 negativas tempranas, las que tienen $Y = 1$. Según el log-odds con prior informativo (Monroe et al., 2008)²¹, la palabra que más separa a las negativas tempranas de las positivas es «not», y por eso la normalización conserva las negaciones. Frente a las tardías aparece lo que pasa al empezar: «tutorial», «account», «login», «settings» y «minutes»; las tardías hablan de «hours», «boss», «mods» y «dlc». El 9.5 % de las negativas tempranas menciona «refund» o una de sus formas, contra el 2.9 % de las tardías y el 0.2 % de las positivas: encaja con la lectura de la señal como proxy, aunque no prueba que esas personas pidieran el reembolso, y la mayoría no lo menciona. El análisis es descriptivo: el texto no entra al modelo de riesgo, y solo lo lee la explicación de motivos, una heurística de palabras clave (sección 7.8). La Parte A prueba si ese lenguaje distingue de verdad a las tempranas (sección 7.9).

5.6. Los 83 contra los 40

Los 40 juegos de la prueba externa pasan las mismas validaciones, pero la mezcla es otra: más juegos gratis (5 contra 2), precios más bajos (una mediana de 269.99 pesos contra 400.00) y una tasa mediana por juego más alta (1.40 % contra 0.80 %), aunque el promedio por juego queda igual, 2.33 % y 2.33 % (detalle en el notebook 00, §3.6). No se decidió nada con esto: los externos nunca entran a elegir variables, a entrenar ni a fijar umbrales. Sí cambia cómo se lee la prueba externa, que mide al modelo con una mezcla de juegos distinta (sección 7.5).

6. Procesamiento de datos

El procesamiento tiene dos destinos. El modelo de riesgo se entrena con data-v1 tal como se publicó, porque solo usa variables del juego y no lee texto. La limpieza produce otro conjunto, el limpio, que usan el análisis de texto (sección 5.5) y la medición de los motivos (sección 7.8).

6.1. Validación y limpieza

Validación. El notebook 00 corre primero los chequeos de la sección 4.3, y los de rango que nunca deben fallar son aserciones²². Además, 57 textos idénticos de 8 palabras o más aparecen en más de un juego, 52 de ellos en folds distintos: un modelo de texto podría verlos en entrenamiento y en prueba, y por eso la limpieza quita los duplicados.

Limpieza: marcar, no borrar. Cada regla es una función de `limpieza.py`. Solo dos quitan filas: los textos idénticos, de los que se queda la reseña más antigua (483 filas), y las

²¹Compara qué tan seguido aparece cada palabra en un grupo y en el otro; el prior evita que una palabra rara parezca distintiva solo por aparecer un par de veces.

²²Comprobaciones dentro del código que detienen la ejecución si no se cumplen.

reseñas vacías (545); quedan 122,944 reseñas. Las demás agregan una columna que marca la reseña y la dejan en su lugar: las cortas, que son el 22.7 % y cuya salida cambiaría qué reseñas se estudian; las escritas en otro idioma, y las plantillas. La normalización pasa el texto a minúsculas, quita el BBCode²³ y las direcciones web, y abre las contracciones para que la negación quede como palabra suelta (detalle en el notebook 00, bloque 2). La limpieza no se aplica al modelo de riesgo. Con la misma partición congelada, sin los duplicados su PR-AUC pasaría de 0.0694 a 0.0695, y sin las vacías, a 0.0696; sin las cortas subiría a 0.0800, pero sobre otra población y con más prevalencia, así que no se compara. El conjunto limpio se guarda con un sha256 de su contenido.

Idioma. La ingesta pidió reseñas en inglés. Según *lingua*²⁴, el 98.7 % de las reseñas limpias está en inglés o no se pudo determinar, y 1,637 están en otro idioma, sobre todo ruso, español y portugués. Como el filtro lo aplicó Steam, esa cifra mide qué tan bien etiquetaron sus reseñas los autores. Las reseñas en otro idioma se marcan y se quedan; al modelo no le afectan, pero a los motivos sí, porque sus palabras clave están en inglés.

6.2. Variables, imputación, valores extremos y aumentación

Ingeniería de variables. El modelo usa cinco variables del juego. El precio entra como el logaritmo de uno más el precio, para que los juegos caros no dominen la escala; la nota de Metacritic entra junto con una bandera que dice si el juego la tiene, y las cinco se estandarizan dentro del modelo (sección 7).

Imputación. La nota que falta se llena con la mediana de *data-v1*, pero siempre junto a la bandera, así que el hueco no se rellena en silencio. Los 2 juegos de pago que llegaron sin precio entran con precio 0: el modelo los lee como si costaran 0, aunque no los marca como gratis, y como el coeficiente del precio es positivo, eso tiende a bajar su riesgo estimado. La ficha de esos juegos lo avisa (sección 7.7). El 0 de los perfiles privados no se imputa como biblioteca vacía: es una bandera de privacidad, y además no entra al modelo.

Valores extremos. No se recorta ninguno. Los rangos imposibles no tienen casos (sección 4.3), el precio ya entra en logaritmo, y de los minutos al reseñar solo importa si quedan antes o después de los 120, así que una reseña escrita tras miles de horas no pesa más que otra.

Aumentación de datos. No aplica. El desbalance de la clase se atiende con pesos de clase en el modelo y con una métrica que no lo esconde (sección 5.2); además, lo que varía es el juego, y crear reseñas sintéticas no agregaría juegos.

²³El formato que Steam permite en las reseñas, con etiquetas como [h1] o [spoiler].

²⁴Una biblioteca de detección de idioma que solo decide cuando tiene confianza.

6.3. Variables que no se conocen al comprar

Una variable solo puede entrar al modelo si se conoce antes de comprar (tabla 3). Las del juego vienen de la tienda y entran. Las del autor de la reseña, no: `num_reviews` cambia con el tiempo y se lee al descargar, y `steam_purchase`, `received_for_free` y `written_during_early_access` solo existen porque la reseña ya se escribió, así que ninguna está disponible al momento de la compra. La biblioteca del autor sí podría declararse antes; se probó, y su aporte cupo en el ruido entre folds (sección 7.2). En el sitio, el perfil se declara en un formulario y sirve para la afinidad: no mueve el riesgo. El texto de la reseña es otro caso: tampoco existe al comprar, pero además sería una fuga²⁵, porque la señal Y sale de esas mismas reseñas, de su voto y de sus minutos, y el texto acompaña a ese voto.

Variable	Cuándo se conoce	¿Entra al modelo de riesgo?
<code>es_gratis</code> , precio (en logaritmo) y descuento	en la tienda, antes de comprar	sí
Si tiene nota de Metacritic, y la nota	en la tienda, antes de comprar	sí; la nota que falta se rellena con la mediana, junto a la bandera
<code>num_games_owned</code> : juegos del autor	en la reseña; en el sitio se declara «compras al año»	no: su aporte cabe en el ruido entre folds, y 0 es un perfil privado
<code>num_reviews</code> : reseñas del autor	al descargar; cambia con el tiempo	no: no está disponible al momento de la compra
<code>steam_purchase</code> , <code>received_for_free</code> y <code>written_during_early_access</code>	en la reseña	no: no están disponibles al momento de la compra
<code>playtime_at_review</code> y <code>voted_up</code>	en la reseña	no: definen Y
Texto de la reseña	en la reseña	no: sería una fuga, porque Y sale de esas mismas reseñas

Tabla 3: Variables candidatas: cuándo se conocen y si entran al modelo de riesgo. Fuente: `construir_features` de `backend/modelado/entrenar_baseline.py` y el notebook 00, §3.5.

7. Modelación y evaluación

7.1. El modelo y sus conjuntos de variables

El modelo es una regresión logística²⁶ con las variables estandarizadas y pesos de clase balanceados²⁷, sin ajuste de hiperparámetros. Se eligió por tres razones. Usa pocas variables, cinco y todas del juego. Sus coeficientes se leen directo, y son los que alimentan

²⁵Hay fuga cuando las variables de un modelo ya contienen la respuesta que debe predecir. Sus métricas salen mejores de lo que serán en uso.

²⁶Un modelo lineal de clasificación: le da un peso a cada variable, las suma y convierte el resultado en un score entre 0 y 1.

²⁷Con `class_weight="balanced"`, las pocas reseñas con $Y = 1$ pesan en el ajuste tanto como todas las demás juntas, para que la clase rara no se pierda.

la ficha: cada factor que muestra el sitio es un coeficiente por el valor del juego (sección 7.7). Y aunque hay 123,972 reseñas, lo que varía son 83 juegos, y con tan pocos un modelo más flexible tendría poco de dónde aprender sin memorizarlos. La comparación entre algoritmos está en la Parte A, sobre el texto de las reseñas (sección 7.9). La misma función, `construir_pipeline`, arma el modelo que sirve la API y el de los notebooks, y se probó con tres conjuntos de variables, siempre contra el clasificador trivial (tabla 4).

Conjunto	Variables	Cuántas	PR-AUC	Desv. entre folds	Veces el trivial
Trivial	ninguna	0	0.0219	0.0037	1.0×
Juego (el del sitio)	gratuidad, precio, descuento y crítica	5	0.0694	0.0415	3.2×
Compra	las de juego y los juegos del autor	6	0.0709	0.0270	3.2×
Completo	las de compra, la bandera de perfil privado y las posteriores a la reseña	11	0.0849	0.0207	3.9×

Tabla 4: PR-AUC del mismo modelo con cada conjunto de variables: media y desviación estándar de los 5 folds de la partición congelada, y cuántas veces la media supera a la del trivial. Fuente: `evaluar_gkf` de `backend/modelado/entrenar_baseline.py` sobre `data-v1`.

El conjunto completo da el PR-AUC más alto, 0.0849, pero no se puede usar: sus variables posteriores a la reseña no están disponibles al momento de la compra (sección 6.3). La comparación que importa es entre juego y compra.

7.2. La decisión B+: no forzar una personalización que los datos no justifican

El proyecto se quedó con el conjunto juego, la variante que llamó B+, por dos razones. La primera es estadística. Compra le agrega al conjunto juego la biblioteca del autor, lo único del jugador que el sitio podría pedir en un formulario, y con eso el PR-AUC sube 0.0015, muy por debajo de la desviación entre folds del modelo (0.0415). Compra le gana a juego en 3 de los 5 folds, y en 16 de 30 particiones distintas de los juegos: más o menos la mitad de las veces, como al azar. La segunda es de diseño. Cuando el sitio servía el conjunto compra, la variable `log_num_games_owned` se llenaba con las compras al año que la persona declara en el formulario, pero el modelo la había aprendido del tamaño de la biblioteca de quien escribió la reseña; las compras declaradas por año eran un sustituto de ese dato que nunca se validó.

Con el conjunto juego, el riesgo es del título y vale igual para cualquier persona: no depende de un dato que nadie verifica y se explica solo con propiedades del juego. El perfil que se declara en el sitio sirve para la afinidad, que dice qué tanto encaja un juego con quien lo declaró, y no mueve el riesgo.

7.3. Validación agrupada por juego

Si las reseñas se repartieran al azar entre folds, el mismo modelo daría 0.0799, con una desviación de apenas 0.0032, porque cada fold de prueba tendría juegos que el modelo ya vio (sección 5.3). Agrupando por `appid` da 0.0694 ± 0.0415 , la cifra que vale para un juego nuevo. El reparto de los juegos también pesa: con 20 particiones por juego hechas al azar, el PR-AUC medio va de 0.0477 a 0.0873. Por eso la partición está congelada en un archivo de `backend/referencias/`, y toda evaluación y los umbrales la leen de ahí. Hay otra razón: `GroupKFold` reparte los juegos según su número de reseñas, 73 de los 83 tienen exactamente el tope de 1,500, y cada versión de `scikit-learn` desempata distinto. Con la 1.6.1, la que trae Colab, solo 12 juegos quedan en el mismo fold; aun así, el PR-AUC es 0.0692 ± 0.0412 y el modelo le gana al trivial en 5 de 5 folds, igual que con la congelada.

7.4. Coeficientes y bandas de riesgo

Los coeficientes de la crítica están en la sección 5.4. Los demás cruzan el cero: el del precio va de -0.01 a 0.94, el del descuento, de -0.31 a 0.07, y el de la gratuidad, de -0.12 a 0.79. El del precio es positivo en el 96.6 % de las réplicas, pero con 83 juegos su intervalo no descarta el cero, y por eso su evidencia se marca como débil.

El sitio no muestra el score: muestra una de tres bandas, bajo, medio o alto. Los cortes son los percentiles 33.3 y 66.7 de los scores fuera de fold²⁸ de las reseñas de entrenamiento, las de `data-v1` (`backend/modelado/entrenar_modelo.py`): un juego pasa a medio desde 0.2858 y a alto desde 0.3916. Los percentiles se toman sobre reseñas, no sobre juegos ni sobre el catálogo, y cada juego se califica después con el modelo entrenado con todo `data-v1`, así que los juegos no se reparten en partes iguales: los 83 de entrenamiento quedan en 30, 23 y 30, y los 123 del catálogo, en 43 de riesgo bajo, 37 de medio y 43 de alto. Por los pesos balanceados, el score no es una probabilidad calibrada: los cortes quedan muy por encima de la prevalencia, que es de 2.19 %. Sirve para ordenar títulos, no como la probabilidad de que una compra termine en la señal. Los umbrales salen solo de `data-v1`, y los 40 externos se clasifican con ellos sin ajustarlos.

7.5. Resultados por fold y prueba externa

La figura 6 muestra el PR-AUC de cada fold de la partición congelada. El modelo le gana al trivial en 5 de 5 folds, pero por márgenes muy distintos, de 1.8 a 5.8 veces; en promedio llega a 0.0694 contra 0.0219, 3.2 veces el trivial. Esa media la carga el fold 1: su PR-AUC, 0.146, es el 42 % de la suma de los cinco, y los otros cuatro promedian 0.0503, cada uno todavía por encima de su trivial. De ahí la desviación tan grande al lado de la media: el resultado depende de qué juegos le tocan a cada fold.

Los 40 títulos que llegaron con `data-v2` se califican con el modelo entrenado en `data-v1`, con sus umbrales y su mediana de Metacritic, igual que lo hace la API. Su PR-AUC es 0.0356 contra 0.0234 del trivial: 1.5 veces, la última pareja de la figura. Con los títulos

²⁸El score fuera de fold (OOF) de una reseña lo da un modelo entrenado sin el juego de esa reseña, con la misma partición congelada.

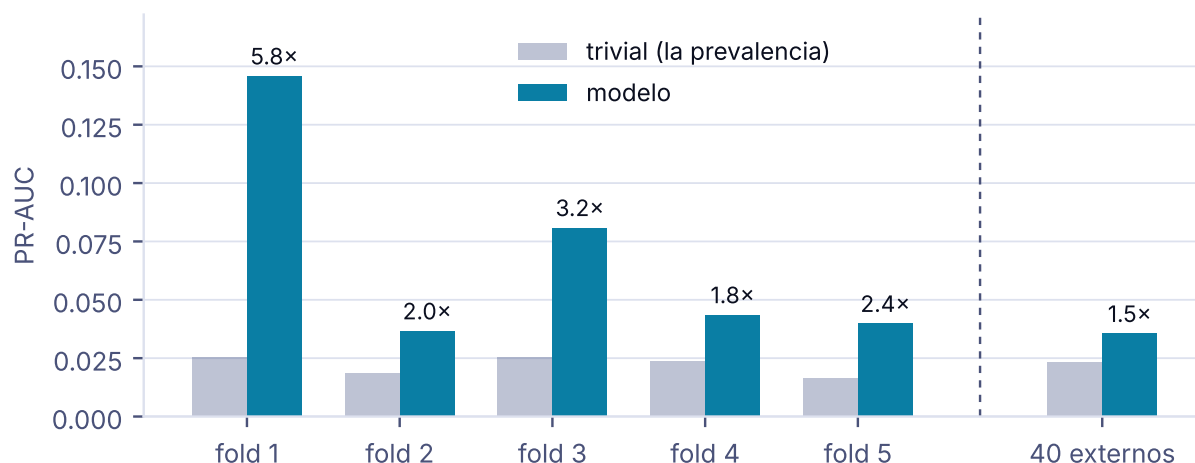


Figura 6: PR-AUC del modelo y del trivial en cada fold de la partición congelada (data-v1) y en la prueba externa. Sobre cada barra, cuántas veces supera el modelo al trivial. Fuente: evaluar_gkf sobre data-v1 y docs/evidencia/prueba-externa.json.

remuestreados 2,000 veces, el intervalo al 95 % del cociente va de 1.03 a 2.33, y solo 27 réplicas quedan en 1 o menos. Hay señal fuera de muestra, pero es modesta: el extremo inferior queda cerca de 1. Entrenar con los 123 juntos bajaría el ruido entre folds, pero se perdería la única evaluación con juegos que el modelo no vio, y por eso el modelo se queda entrenado con data-v1.

7.6. Por qué cae fuera de muestra

La tabla 5 da la tasa de la señal en cada banda. En data-v1, cada juego se clasifica con su score fuera de fold, el de un modelo que no lo vio, y por eso sus bandas tienen 27, 27 y 29 juegos y no los 30, 23 y 30 de la banda que sirve el sitio, que sale del modelo entrenado con todo data-v1. Los externos se clasifican con ese modelo final, que tampoco los vio, así que su banda es la servida. Con el score fuera de fold, la tasa de la banda alta es 6.3 veces la de la baja; en los externos, 1.9 veces, y la media no se separa de la baja. En un título nuevo, lo que se sostiene es la banda alta.

Conjunto	Banda	Juegos	Reseñas	Con señal	Tasa de señal	Veces la base
data-v1, OOF	bajo	27	40,698	283	0.70 %	0.3×
	medio	27	40,599	565	1.39 %	0.6×
	alto	29	42,675	1,866	4.37 %	2.0×
40 externos	bajo	13	19,698	366	1.86 %	0.8×
	medio	14	21,098	344	1.63 %	0.7×
	alto	13	19,599	702	3.58 %	1.5×

Tabla 5: La señal por banda de riesgo. En data-v1 la banda sale del score fuera de fold (OOF) y no es la que sirve el sitio; en los externos es la servida. La tasa es la parte de las reseñas con la señal, y «veces la base» la compara con la tasa de todo el conjunto. Fuente: la salida guardada de lr.tabla_de_bandas en el notebook 01 (codigo-v6), sin recalcul.

Una parte de la caída se ve en la variable que más pesa, la crítica: de los 43 juegos de riesgo alto del catálogo, 33 no tienen nota, y en los títulos nuevos, que traen otra mezcla, la falta de nota separa menos (sección 5.6; detalle en docs/evidencia/metacritic-por-banda.md).

Las bandas cortan un score continuo, y un juego cerca de un corte puede cambiar de banda con un cambio mínimo en los datos o en el reentrenamiento. Los dos más cercanos son Hollow Knight, con 0.2857 contra el corte de 0.2858, en la banda baja, y Warframe, con 0.3918 contra 0.3916, en la alta. Por eso no se usan como casos de demostración.

7.7. Cómo se explica el riesgo

Para cada juego, la API calcula el aporte de cada variable al score, su coeficiente por su valor estandarizado²⁹, y la ficha muestra los tres de mayor aporte, aunque el primero tenga evidencia débil. Cada factor lleva su nivel de evidencia, según sus intervalos (sección 7.4): sólida para la cobertura y la nota de la crítica, y débil para la gratuidad, el precio y el descuento, con la leyenda «evidencia débil: con 83 juegos no se distingue de cero». Junto a cada factor va la cifra del juego y su referencia: la nota contra el promedio del catálogo, 85.5, y el precio contra la mediana de los de pago, 349.88 pesos, que se parece más a la escala logarítmica del modelo que el promedio.

Un factor con un aporte menor que UMBRAL_TIPICO, 0.10, se muestra sin flecha, como «cerca de lo típico del catálogo». Ese valor es una decisión de producto, no una propiedad estadística: no sale de un intervalo ni de una prueba. Se eligió para que ninguna flecha contradiga la cifra que tiene al lado, porque el modelo compara cada nota contra su media de entrenamiento, 86.97, mientras que la ficha la compara contra el promedio del catálogo; una nota entre las dos aparece «por encima del promedio» y aun así recibe un aporte pequeño que sube el score. Con 0.10, ningún factor con flecha cae en ese caso en los 123 juegos, y el generador de este documento lo comprueba.

Dos casos llevan aviso. En un juego gratis, la gratuidad y el precio de 0 dicen lo mismo y tiran en sentidos opuestos, así que se muestran como un solo factor, y el aviso agrega que en el entrenamiento había solo 2 juegos gratis y que, para ellos, el modelo extrapola. En los 2 juegos de pago sin precio, el aviso dice que la estimación es menos confiable, porque el modelo tomó el precio como 0.

7.8. Los motivos: una heurística de palabras clave

/explicacion no usa un modelo. Busca palabras clave en inglés de 6 categorías (rendimiento, bugs, dificultad, controles, contenido y precio) en las reseñas con la señal de cada juego, y una reseña puede caer en varias o en ninguna. Si el juego tiene menos de 5 reseñas con la señal, no muestra motivos, y cada porcentaje se calcula sobre las reseñas que sí cayeron en alguna categoría, cuyo número la ficha dice. El límite es la cobertura: de las 2,706 negativas tempranas del conjunto limpio, el 34.0 % menciona al menos un motivo de la lista, y el 66.0 % restante no menciona ninguno; rendimiento y bugs son los

²⁹El aporte está en log-odds, la escala en la que la regresión suma: positivo sube el score y negativo lo baja.

más frecuentes (detalle en el notebook 00, §3.8). Por eso los motivos se leen como las quejas de las reseñas que usan esas palabras, no como la causa de la señal en ese juego.

7.9. Modelos de texto (Parte A)

Los motivos de la ficha se cuentan en las negativas tempranas. Para presentarlos como quejas de las primeras horas, y no como quejas generales, esas reseñas tendrían que hablar distinto de las tardías. La Parte A lo pone a prueba con modelos que leen el texto: entre las negativas, ¿se distinguen las tempranas de las tardías? Es un análisis aparte del modelo de riesgo, y su resultado no entra al score: la señal sale de esas mismas reseñas, y usar su texto para estimarla sería fuga.

El prerregistro. Los modelos, la métrica y la regla de decisión se fijaron por escrito³⁰ en el commit 6bbcea6, anterior al del código, 59694a6, y el resultado salió con el árbol de git limpio (docs/evidencia/modelos-texto.json). La regla descarta el mejor modelo si el intervalo de su cociente contra el trivial incluye el 1, y también si no le gana a una regla de una sola palabra: marcar las reseñas que dicen «refund». Esa palabra salió de mirar estos mismos datos (sección 5.5), así que la referencia es optimista.

Datos y modelos. Son las 16,836 reseñas negativas en inglés de los 83 juegos de data-v1, con la partición congelada; 2,670 son tempranas, el 15.86 %. Una reseña que dice cuánto jugó repite la etiqueta, así que en las 2,442 que escriben una duración esta se cambia por una marca. Se compararon tres modelos: TF-IDF³¹ con Naive Bayes (NB), TF-IDF con regresión logística (LR) y MiniLM, un modelo de lenguaje preentrenado y congelado, con la misma regresión logística.

Resultado. TF-IDF + LR es el mejor (figura 7): llega a 0.369, con un intervalo de 0.288 a 0.421, contra 0.161 del trivial, 2.3 veces, de 2.05 a 2.53. A la regla «refund», que llega a 0.185, le saca 0.184, de 0.137 a 0.205, y en la comparación pareada también queda arriba de NB (0.226) y de MiniLM (0.323). Es la rama 3 del prerregistro: las quejas tempranas tienen lenguaje propio, más allá de mencionar el reembolso. Se documenta TF-IDF + LR, que no se sirve en ningún lado.

Lectura para negocio. Se hizo después de la decisión, con los mismos scores fuera de fold, y es descriptiva. Si alguien leyera solo la décima parte de las negativas, las que TF-IDF + LR señala primero, el 43.0 % serían tempranas, 2.7 veces la prevalencia. Las nubes (figura 8) dicen de qué se habla de cada lado. Hacia temprana pesan el reembolso, lo que pasa al abrir el juego (el tutorial, la cuenta, el inicio de sesión) y el rendimiento; hacia tardía, lo de quien ya jugó mucho: los mods, las actualizaciones, los jefes. De las 40

³⁰Un prerregistro fija, antes de correr el análisis, qué se va a medir y qué resultado contará como éxito, para no acomodar la pregunta a los resultados.

³¹Representa cada reseña por sus palabras y pares de palabras, con menos peso para las que aparecen en casi todas.

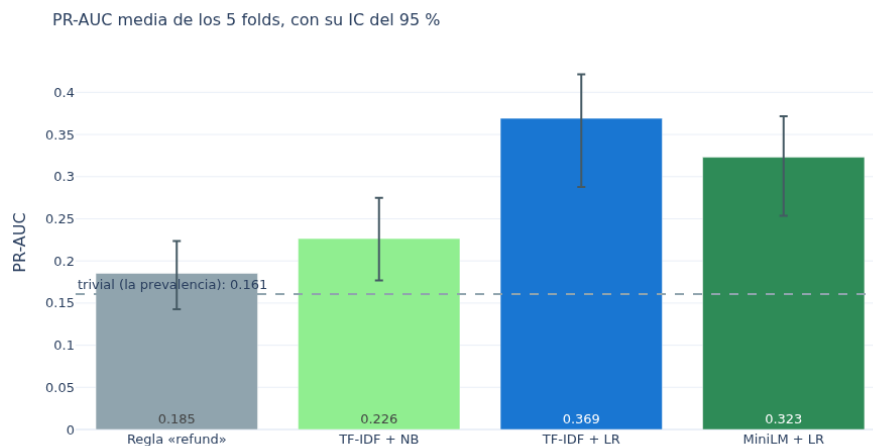


Figura 7: PR-AUC de los modelos de texto: media de los 5 folds de la partición congelada, con su intervalo al 95 % por bootstrap sobre juegos. La línea punteada es el trivial. Fuente: la salida guardada del notebook 02 en código-v6.

palabras que más empujan hacia temprana, 35 no caen en ninguna de las 6 categorías de motivos del sitio.

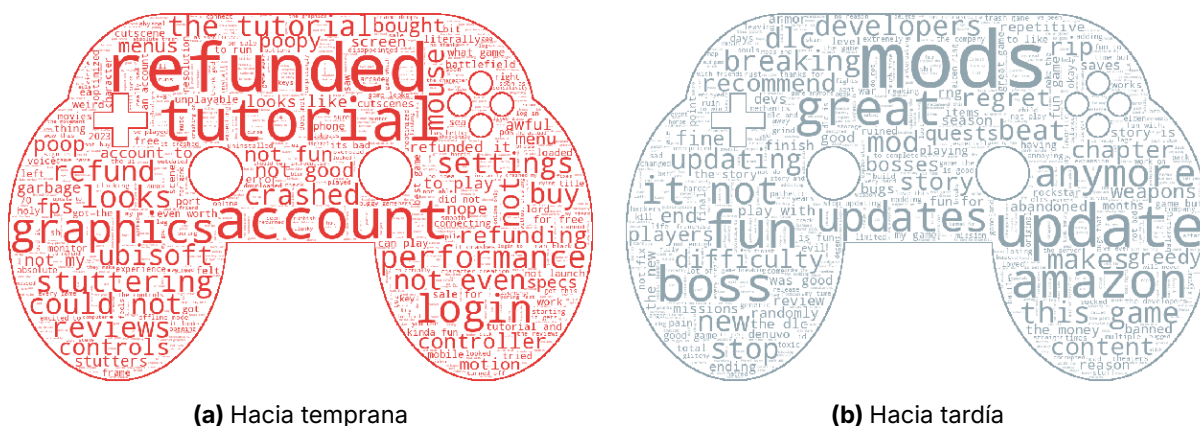


Figura 8: Las palabras que empujan a TF-IDF + LR hacia cada clase en los 5 folds; el tamaño sigue a su coeficiente medio. Son asociaciones del modelo, no causas. Fuente: la salida guardada del notebook 02 en código-v6.

Lo que no dice. Es la señal en el texto de quien escribió pronto, no un arrepentimiento confirmado. Que el modelo distinga las tempranas tampoco prueba qué categorías de motivos funcionan: eso es la Parte B (sección 9.3). El análisis cubre solo reseñas en inglés de data-v1, y su intervalo no incluye la variación que daría reentrenar.

8. Resolución del problema

8.1. Arquitectura y despliegue

La figura 9 sigue el camino de los datos. La ingesta bajó las reseñas y las fichas de Steam y las publicó en releases con tag fijo (sección 4.2); de ahí en adelante nada vuelve a consultar Steam. Render construye la API con backend/despliegue/Dockerfile, y el build falla si algo no cuadra: `preparar_entorno.py` se detiene si el sha256 de data-v3 o de data-v1 no es el publicado, entrena el modelo con data-v1 y la partición congelada, y `verificar_bandas.py` compara la banda de los 123 juegos con la referencia validada, así que basta con que cambie una para que falle. El build también rechaza la imagen si le llega una biblioteca de modelos de texto, que deben calcularse fuera de la API.

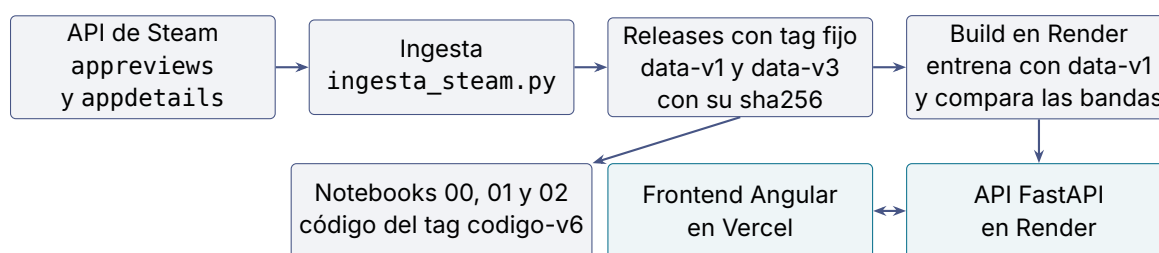


Figura 9: De la API de Steam a la interfaz. Después de la ingesta, ningún paso consulta Steam. Fuente: backend/despliegue/Dockerfile, `preparar_entorno.py` y `frontend/vercel.json`.

La API, hecha con FastAPI, expone 17 endpoints: 5 de riesgo (`/catalogo`, `/perfil`, `/prediccion`, `/explicacion` y `/panorama`), 8 para calificaciones, comentarios y reacciones, y 4 de Nia, descritos en su página `/docs`³². Todo lo que entra se valida con Pydantic³³. `/prediccion` sigue recibiendo el perfil por compatibilidad, pero el riesgo que devuelve es del título y no cambia con él. El frontend, en Angular, es un sitio estático en Vercel.

La API está en el plan gratis de Render: se duerme cuando nadie la usa, y la primera consulta después tarda en responder. Su disco es efímero, así que las calificaciones, los comentarios y los votos a Nia se borran al reiniciarse; el catálogo y el modelo vienen dentro de la imagen y no se pierden. Nia usa un modelo de lenguaje de OpenAI solo si el servidor tiene clave; si no, contesta por reglas, sin costo. Los comentarios, las reacciones y las consultas y votos a Nia tienen un límite por minuto, por usuario y por IP.

8.2. Interfaz de uso

Una consulta sigue estos pasos:

1. En el Inicio, la persona escribe el nombre de un juego en el buscador.

³²FastAPI arma esa página con el contrato OpenAPI de cada endpoint: qué recibe, qué devuelve y qué errores da.

³³Una biblioteca de Python que revisa el tipo y el rango de cada dato de entrada y rechaza la solicitud si no cumple.

2. La ficha del juego da la banda con su frase, los tres factores que la mueven con su evidencia, los motivos más frecuentes con el número de reseñas detrás y el precio al corte de datos, con el enlace a Steam. Si la persona llenó su perfil, la ficha dice además qué tanto encaja el juego con él; eso no cambia la banda.
3. Desde la ficha agrega el juego a Comparar, que lo pone junto a otros.
4. En Explorar recorre los estantes por banda y filtra por género para buscar alternativas.
5. A Nia le pregunta lo que no encuentra en la ficha, como por qué un juego tiene esa banda o cuál de dos le conviene.

El sitio tiene además el Perfil, el Historial de consultas, el Panorama del catálogo y una página que explica cómo se calcula la estimación. La sección 3.3 muestra estos pasos con capturas.

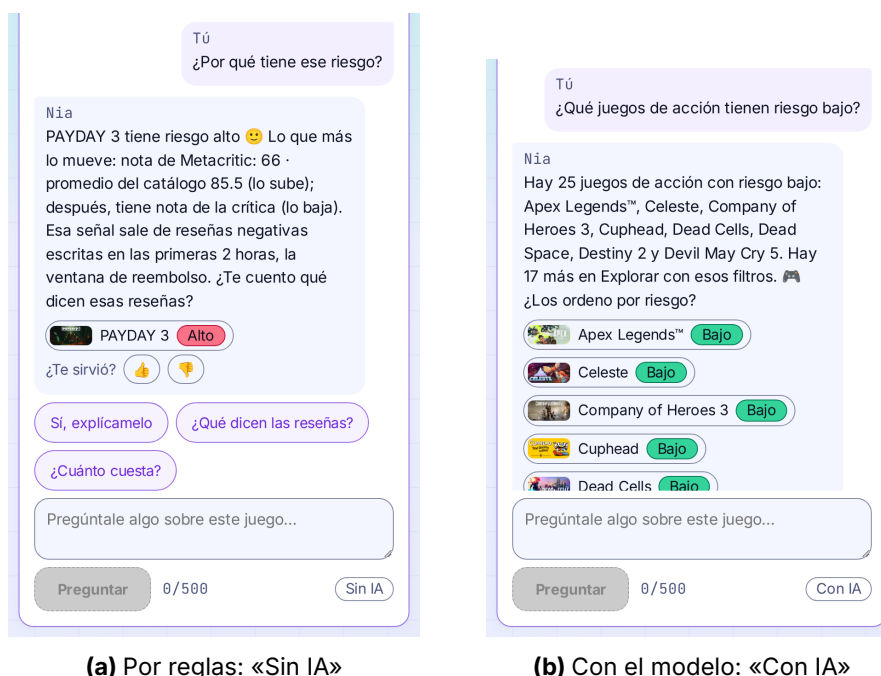


Figura 10: Nia en la ficha de PAYDAY 3. (a) «¿Por qué tiene ese riesgo?» se contesta por reglas aunque haya modelo de lenguaje; captura provisional del sitio, que se regenera desde el tag de entrega. (b) «¿Qué juegos de acción tienen riesgo bajo?» la contesta el modelo de lenguaje; captura de producción, en `nexplay-six.vercel.app`.

Nia es el asistente del sitio (figura 10). Con un modelo de lenguaje, no recibe el catálogo en el texto de la conversación: usa 5 herramientas que leen lo mismo que sirve la API, como buscar juegos, abrir la ficha de uno o consultar el panorama, y nunca las calificaciones ni los comentarios de la gente. Cada juego que menciona tiene que salir de esas herramientas, y la respuesta se valida contra ellas antes de mostrarse. Las preguntas que deben contestarse siempre igual, como «¿por qué tiene ese riesgo?», un juego fuera del catálogo o cualquier tema ajeno a los juegos, van por reglas aunque haya modelo, y sin clave de OpenAI todo va por reglas. Una etiqueta junto al chat dice quién escribió la última respuesta: «Con IA» si fue el modelo de lenguaje y «Sin IA» si fueron las reglas.

(`frontend/src/app/chat/nia.ts`; figura 10). Si la pregunta trae algo que parece un correo o un teléfono, se borra antes de anotarla: Nia no guarda datos personales.

Pruebas. `make test` corre el verificador de bandas, el de Nia, el de los factores de la ficha, el de los niveles, el de `preparar_entorno.py` y las pruebas del frontend. El de los niveles (`verificar_niveles.py`) comprueba que la API, el frontend y Nia usan la misma definición de bajo, medio y alto, la de los percentiles de la sección 7.4, y que ningún texto de la API ni del frontend vuelve a la definición anterior, la de tercios del catálogo. Nia se probó, además, con su modelo de lenguaje: pasó 25 de las 25 preguntas de regresión, y en la corrida del 30 de septiembre de 2026 contra 48 preguntas trampa y legítimas quedaron 0 casos por revisar³⁴. La prueba local del camino con IA es `verificar_nia.py` - `-openai`, que manda el recorrido de preguntas al modelo de lenguaje de OpenAI: corrió el 2 de octubre de 2026 sobre el código del tag `codigo-v7` y terminó sin problemas³⁵.

8.3. Cómo responde la pregunta

La pregunta de datos era qué tienen en común los juegos que dejan más señal y si se puede anticipar antes de comprar. La respuesta es sí, en parte. La señal cambia mucho de un juego a otro (sección 5.3), y lo que mejor la acompaña es la crítica: no tener nota de Metacritic, o tener una baja (sección 5.4). Con eso, el modelo ordena los juegos que no vio 3.2 veces mejor que el trivial en la validación agrupada y 1.5 veces mejor en la prueba externa, con un intervalo de 1.03 a 2.33 (sección 7.5). Hay señal, y es modesta.

La pregunta de producto era cómo mostrarlo sin prometer una probabilidad. El sitio da una banda con palabras, la explica con factores que dicen su evidencia, y suma los motivos de las reseñas, que son una heurística de palabras clave con su cobertura a la vista (sección 7.7). Por la Parte A, esos motivos se pueden presentar como quejas propias de las primeras horas y no como quejas generales (sección 7.9). Con eso la persona decide: si compra, si compara antes con otro juego, si busca una alternativa de riesgo bajo en el mismo género, o si compra y usa las dos primeras horas para probar justo lo que más se queja la gente, mientras todavía puede pedir el reembolso. El riesgo es el mismo para cualquier persona; lo que cambia de una a otra es qué quejas le afectan y qué tanto encaja el juego con lo que declaró.

9. Conclusiones

9.1. Limitaciones

- **Es una proxy.** La señal es una reseña negativa temprana, no el arrepentimiento ni el reembolso: deja fuera a quien no escribe y a quien escribe después de las dos horas (sección 5.1).

³⁴El detalle está en `docs/evidencia/nia-pruebas.md`. La corrida de trampas se repite sobre el tag de entrega.

³⁵La salida completa está en `docs/evidencia/verificar-nia-openai-2026-10-02.txt`.

- **La muestra retrata el presente.** El 96.1 % de las reseñas de entrenamiento es de julio de 2025 en adelante, así que la señal de un juego viejo viene de quien lo compra tarde (sección 4.2).
- **Hay pocos juegos.** El modelo aprende de 83, y el resultado cambia mucho de un fold a otro. Con solo 2 juegos gratis, el modelo extrapola en los gratis, y los 2 juegos de pago sin precio entran con precio 0; los dos casos llevan aviso en la ficha.
- **En un título nuevo, solo la banda alta se sostiene.** En los externos, la banda media no deja más señal que la baja (sección 7.6).
- **El orden por score también lo arrastra el precio 0.** En el estante de riesgo bajo de Acción, los primeros lugares son de un juego sin precio y de dos gratis, todos con aviso; la primera alternativa sin aviso es la tercera tarjeta (sección 3.3).
- **Los motivos son palabras clave en inglés.** Solo el 34.0 % de las 2,706 negativas tempranas del conjunto limpio menciona alguno (sección 7.8), y la mayoría de las palabras que más empujan hacia temprana no cae en ninguna categoría (sección 7.9).
- **Representatividad.** Las reseñas son en inglés y las escriben compradores de todo el mundo, mientras que los precios son de la tienda de México y el usuario objetivo compra desde México. Tampoco hay datos de PlayStation ni de Xbox.
- **No hubo un estudio con usuarios externos.** La usabilidad se apoya en decisiones de diseño y en pruebas automáticas, no en sesiones con personas que no conocían el sitio.
- **La infraestructura es gratuita.** La API se duerme cuando nadie la usa y su disco borra las calificaciones y los comentarios al reiniciarse (sección 8.1).

9.2. Qué se concluye

La señal de arrepentimiento temprano depende sobre todo del juego: la varianza de su tasa entre juegos es unas 107 veces la esperada si todos tuvieran la misma tasa, y lo que mejor la acompaña es la crítica. Un modelo pequeño, una regresión logística con cinco variables que se conocen antes de comprar, ordena los juegos que no vio mejor que el trivial: 3.2 veces en la validación agrupada y 1.5 veces en la prueba externa, con un intervalo que no baja de 1.03. Es una ventaja real y modesta, y el documento la reporta siempre junto al trivial.

El perfil de quien compra no la mejora más allá del ruido entre folds, y el dato que lo representaba en el sitio no estaba validado. Por eso el riesgo es del título y vale igual para cualquier persona. El aporte del proyecto es convertir esa ventaja en algo que sirve antes de pagar: una banda con palabras y no una probabilidad, factores que dicen qué tan firme es su evidencia, motivos con su cobertura a la vista y acciones concretas, como comparar, buscar una alternativa o aprovechar la ventana de reembolso. El texto de las reseñas agrega algo que el modelo de riesgo no da: las quejas tempranas tienen un lenguaje propio, más allá de mencionar el reembolso (sección 7.9), y eso respalda presentar los motivos como quejas de las primeras horas. Lo que no hace es decirle a nadie qué tan probable es que se arrepienta, ni explicar con certeza por qué un juego decepciona.

9.3. Siguiendo pasos

La Parte B es trabajo futuro y no ha empezado. Arranca de la cobertura de los motivos: probar si un modelo que lee el texto completo asigna motivos a más negativas tempranas que las palabras clave, contra un conjunto de referencia etiquetado a mano, con su propio prerregistro. El notebook 02 propone un paso más corto, ampliar las categorías con lo que hoy queda fuera, como la cuenta y el acceso o los primeros pasos, y también lo deja para un prerregistro propio, porque cambia lo que ve la gente. Más allá de eso, el modelo ganaría con más juegos, en especial gratis, y con una prueba de la interfaz con personas que no la conocen.

Referencias

- Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. *Proceedings of the 23rd International Conference on Machine Learning*, 233-240. <https://doi.org/10.1145/1143844.1143874>
- Monroe, B. L., Colaresi, M. P., & Quinn, K. M. (2008). Fightin' Words: Lexical Feature Selection and Evaluation for Identifying the Content of Political Conflict. *Political Analysis*, 16(4), 372-403. <https://doi.org/10.1093/pan/mpn018>
- Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Valve Corporation. (s.f.-a). *IUserReviewsService*. Consultado el 30 de septiembre de 2026, desde <https://partner.steamgames.com/doc/webapi/IUserReviewsService>
- Valve Corporation. (s.f.-b). *User Reviews – Get List* [Consultada el 30 de septiembre de 2026; la página marca el endpoint como obsoleto y remite a *IUserReviewsService*]. Consultado el 30 de septiembre de 2026, desde <https://partner.steamgames.com/doc/store/getreviews>
- Valve Corporation. (2024, 23 de abril). *Steam Refunds*. Consultado el 30 de septiembre de 2026, desde https://store.steampowered.com/steam_refunds/
- Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158), 209-212. <https://doi.org/10.1080/01621459.1927.10502953>

A. Reproducibilidad

Cada cifra de este documento sale de un release con tag fijo o de docs/evidencia/, a través de documento/generar_figuras.py, y make doc lo reconstruye todo. La tabla 6 resume los releases y la tabla 7 da el sha256 de cada archivo, que el código compara antes de usarlo.

El código. Todo el Python vive en backend/: la API, el modelado, el análisis, las referencias, el despliegue, los verificadores, la ingesta, la publicación y la operación. El frontend vive en frontend/, los notebooks en notebooks/, este documento en documento/,

Release	Publicado	Juegos	Reseñas	Base (MB)	Extracto (MB)	Para qué
data-v1	2026-09-16	83	123,972	11.4	1.4	Entrenar, elegir las variables y fijar los umbrales.
data-v2	2026-09-21	123	184,367	16.7	1.9	Suma 40 juegos que el modelo no ve: la prueba externa.
data-v3	2026-09-27	123	184,367	16.7	—	data-v2 más los totales públicos de Steam por juego. Lo sirve la API.

Tabla 6: Releases de datos. La base es la copia reproducible de la base de datos (`nexplay_reproducible.db.xz`); el extracto, la tabla sin texto que lee el notebook 01 (`nexplay_extracto.parquet`). Fuente: API de releases de GitHub y los archivos bajados.

Release	Archivo	sha256
data-v1	extracto	645d685df884c9a352e4306e899c62768ea493375d4b0378c201c7fc6d2d2d6e
data-v1	base	2ef8ef40330385af4c03cd072dccb20fc9a4b635e3929e513235c191d14e9ee7
data-v2	extracto	025323e0d0e99031ed940e0a12218ed8e5dd9068bd4b0aedb2a805b01be2a0d1
data-v2	base	9d5a54f6cbb5f361e397eb043989e592cbff1d553c57ef8a41aae41e2c763d72
data-v3	base	44f6704d469a8f59683be8e28429108b7a88a06ee01aecdb21da94d90077dfb8

Tabla 7: El sha256 de cada archivo de los releases, según GitHub. El generador falla si no es el mismo que usa el código.

y la evidencia, las capturas y el historial en `docs/`. Los tres notebooks clonan el tag `codigo-v6`, el commit `94838c2`, y en ese tag guardan las salidas de una corrida local, para que sus gráficas se vean también en GitHub. De esas salidas, sin recalcular, salen la tabla 5 y las figuras 7 y 8.

La corrida en Colab. Los tres notebooks corrieron completos en Colab el 2 de octubre de 2026, con ese mismo tag: 69 de 69 celdas de código en el 00, 28 de 28 en el 01 y 24 de 24 en el 02, sin errores ni avisos, con Python 3.13.15, scikit-learn 1.6.1, numpy 2.1.3 y pandas 2.2.3. Las tres descargas están en `docs/evidencia/colab/`. El generador de este documento comprueba su sha256 y que cada celda de la que toma una cifra haya impreso en Colab lo mismo que en el tag. Con esas versiones, fuera de Colab, el 00 tarda 192 segundos; el 01, 34, y el 02, 400.

Los comandos. Desde la raíz del repositorio, `make setup` crea el entorno de Python e instala el frontend; `make data` baja `data-v3` y `data-v1` y verifica su sha256; `make train` entrena el modelo con `data-v1` y compara las 123 bandas con la referencia; `make api` y `make web` levantan la API y el frontend en local; `make test` corre los verificadores del backend y las pruebas del frontend; `make notebooks` ejecuta los tres notebooks en una carpeta temporal y compara sus salidas con las guardadas, sin tocarlas, y `make doc` genera las cifras, las tablas y las figuras, y compila este documento.